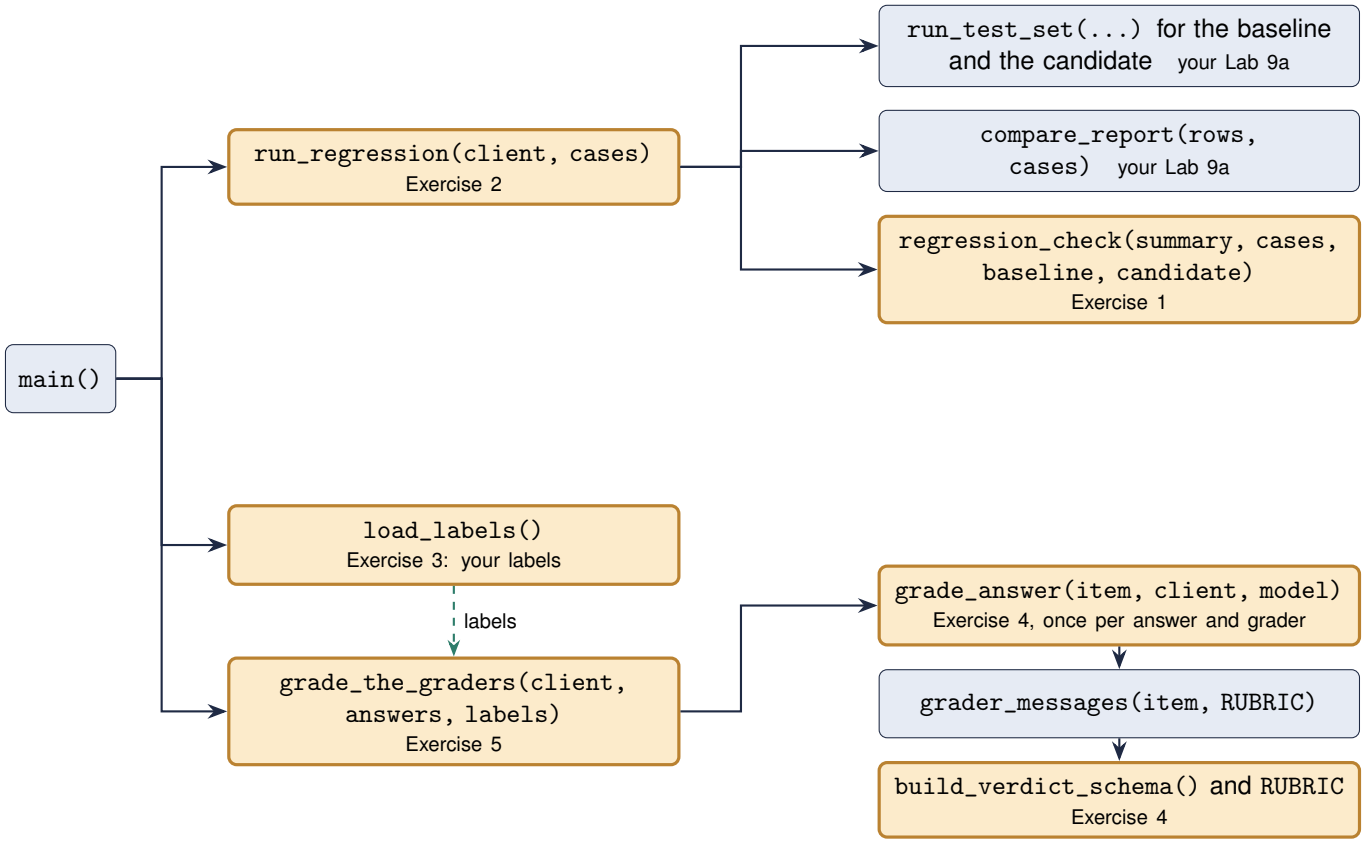
Lab 9b call flow, 1 of 5: who calls whom

`python3 scaffold.py` does two jobs: is the change safe, and does the grader agree with you? Gold boxes are your exercises.



Job 1 reuses Lab 9a. *The only new code is the comparison: which cases did the candidate do worse on? The candidate is `"plan-execute-on-aux"` in `PATTERNS`: your planner, on the smaller model.*

Job 2 needs your labels first. *`main` loads them before any grader runs. Your labels are the answer key the graders are measured against, the same idea as Lab 9a's test set.*

given code | you write this | outside your code | data passed along | calls | returns

Lab 9b call flow, 2 of 5: `regression_check`

Exercise 1. It compares pass *rates*, case by case. A real run, 4 October: your planner on `main`, then on `aux`.

Case	baseline (`main`)	candidate (`aux`)	
escalate-never-escalated	1/1	1/1	ok
known-path	1/1	1/1	ok
older-than-six-days	1/1	0/1	**REGRESSION**
resolved-ticket	1/1	1/1	ok
already-escalated	1/1	0/1	**REGRESSION**
missing-ticket	1/1	0/1	**REGRESSION**
younger-than-six-days	1/1	0/1	**REGRESSION**
only-if-unresolved	1/1	0/1	**REGRESSION**

Read from the summary. *`summary[baseline][case]` and `summary[candidate][case]` each hold `passed` and `runs`. Compare `passed` / `runs`, not raw counts: the two may have run different numbers of times.*

Prediction 2, checked. *Every regressed run on `aux` used all 10 rounds and ended `exhausted`. Open one of its trace files and see what the planner kept doing.*

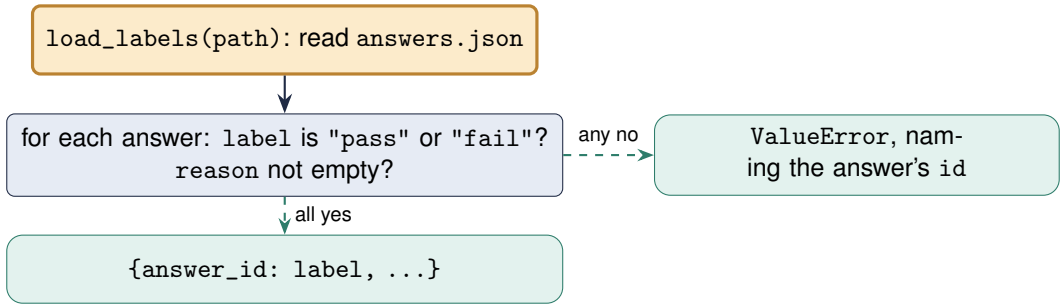
A higher total can still hide a regression. *That is why the check is per case, in the order of `cases`, and why one worse case is enough to say "do not ship".*

given code | you write this | outside your code | data passed along | → calls | - - → returns

Lab 9b call flow, 3 of 5: your labels come first

Exercise 3. You decide pass or fail for each answer before any grader runs. Your loader refuses unfinished labels.

```
{"id": "r1",
 "question": "Ticket 23 is unresolved, 8 days old, ... Should it be
escalated?",
 "answer": "...",
 "label": null,   ← you write "pass" or "fail"
 "reason": ""}   ← and one line saying why
```



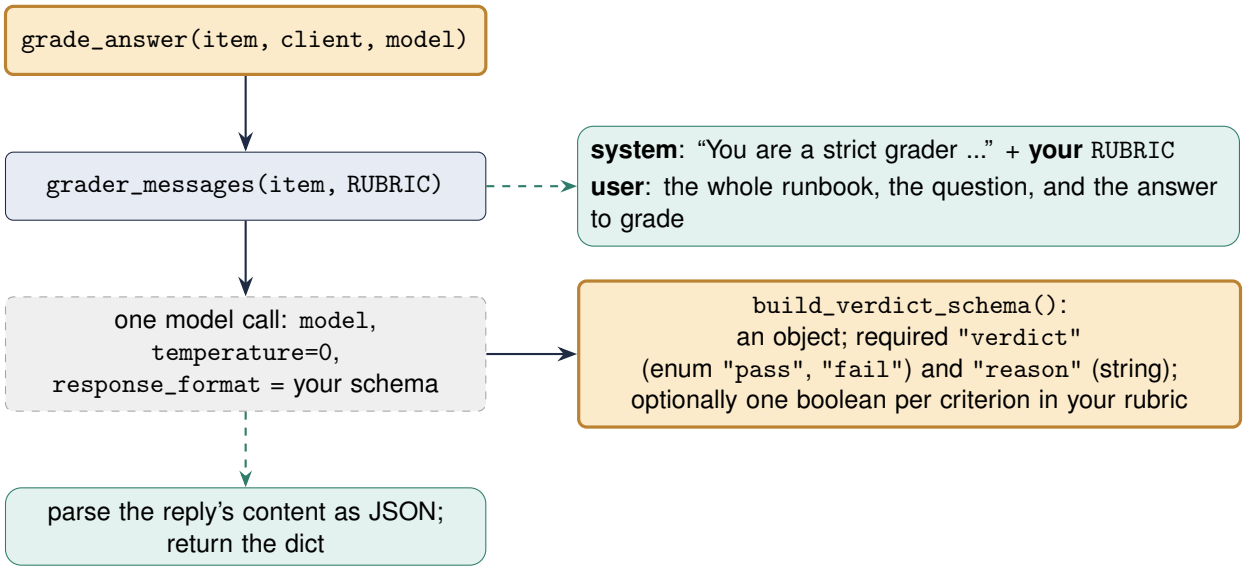
Judge against `runbook.json`, not your own knowledge. *Pass only if every claim agrees with the runbook, nothing is added, and the question is answered or honestly declined.*

Why labels first? *If you saw the grader's verdicts first, you might just agree with them. Then measuring the grader against you would measure nothing.*

given code | you write this | outside your code | data passed along | → calls | - - → returns

Lab 9b call flow, 4 of 5: one call of `grade_answer`

Exercise 4. What the grader is sent, and what it must send back.



The grader sees the reference. *The whole runbook is in the user message, so it can check claims against the source instead of its own memory.*

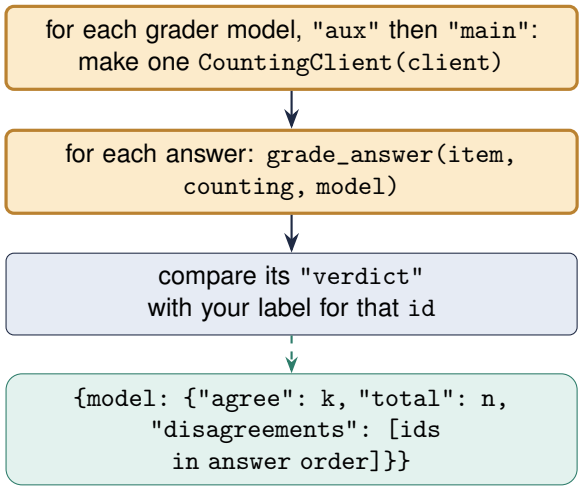
Your rubric is the grader's instructions. *Name each criterion and say exactly when the verdict is pass. A vague rubric makes a vague grader.*

The schema makes the verdict checkable. *Your code reads `"verdict"` directly; no parsing of free text, as in Week 4.*

given code | you write this | outside your code | data passed along | → calls | - - → returns

Lab 9b call flow, 5 of 5: `grade_the_graders`

Exercise 5. The grader gets an evaluation of its own: your labels are its test set.



Same shape as Lab 9a: test cases with answers written first (your labels), a check (does the verdict match?), and a count. Only the thing under test changed: now it is the grader.

***Prediction 3**: the smaller grader agrees with you less often. For each disagreement, re-read the answer: is the grader wrong, or is your label?*

given code | you write this | outside your code | data passed along | → calls | - - → returns